Griffith UNIVERSITY
Queensland, Australia

2802ICT Intelligent Systems

Assignment 2 - Programming

- Decision Tree Learning and
- Neural Networks

- **Due:** Tuesday, 7th October 2025, 11:59 PM.
- **Total Marks:** 40%:

✓ Task 1: 20 marks

✓ Task 2: 20 marks.

School of ICT, Griffith University
Trimester 1, 2025

Instructions:

- **Late submission:** Late submission is allowed but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each working day that the item is late. A working day is defined as Monday to Friday. Assessment items submitted more than five working days after the due date will be awarded zero marks.

- **Extension:** You can request for an extension of time on one of two grounds: medical or special circumstances ((e.g. special family or personal circumstances, unavoidable commitments).

  All requests must be made through the myGriffith portal by providing supporting documents.

- **Individual Work:** You must complete this assignment individually and submit an authentic work. This should be your own work. Anyone caught plagiarising will be penalised and reported to the university.

- **Student Conversation:** Students may be invited for a 10-minute conversation, which will be held immediately after the assignment is submitted (in Week 12 or Week 13). Please be reminded that an assignment will NOT be assessed if a student is invited for the conversation but fail to attend the conversation.

Objectives:

✓Implement learning algorithms.

✓Evaluate the performance of a simple learning system on a real-world dataset.

Your code should be written in Python. In rare cases students may be allowed to use another programming language, but approval by the course convenor is required.

Note 1. You are allowed to use libraries for data structures, math functions and the libraries mentioned in the task description below.

Note 2. You may find some Python packages/functions are useful, such as csv.reader, random.randrange, matplotlib.deepcopy, and copy.deepcopy.

Note 3. You are NOT ALLOWED to use libraries such as sklearn, pytorch, tensorflow to implement the algorithms.

Task 1 – Learning Decision Trees

Implementation Requirements

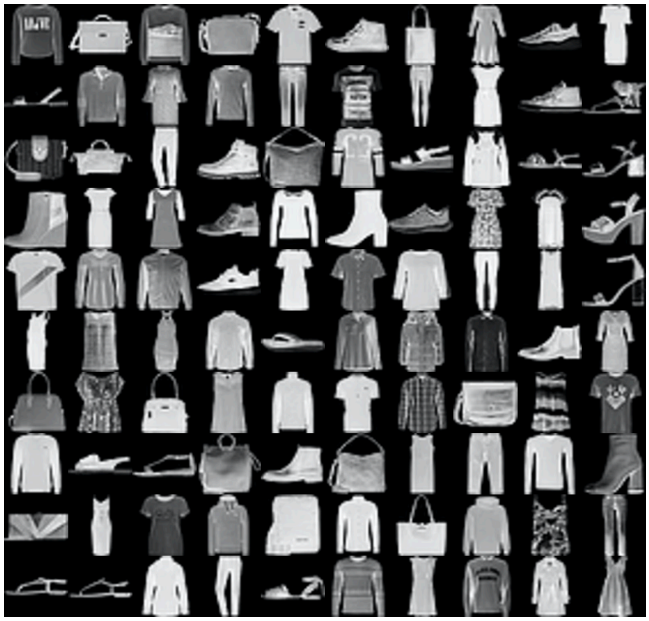
a. Read the **car.csv** dataset, which is designed to evaluate a car. The dataset includes a list of 1728 cars with 6 features: buying price, cost of maintenance, number of doors, capacity in terms of persons to carry, the relative size of luggage boot and the estimated safety value of each car. The **last column** includes the car evaluation, classifying the cars into 4 classes: *unacceptable*, *acceptable*, *good* or *very good*. This task is to learn a decision tree that predicts a given car is in one of these four classes.
b. You should be able to randomly split the dataset into testing and training set.
c. Use the training set (train data) to build a decision tree for predicting that a car is in one of the four classes using the above six attributes.
d. Test your classifier and print:
    1. The size of the training and testing sets
    2. Total accuracy
    3. Precision, recall and F1-score values for each class, together with the macro-average and weighted-average
    4. Plot the Learning curve (accuracy as a factor of percentage of learning examples) i.e. show how the accuracy changes while learning.
       For example, if the training set includes 1000 samples, you may stop building the tree after 100 samples and test with the current tree, then continue building the tree and stop again to test after 200 samples, and so on until you have the final decision tree which is built using the full training set.

Note: To make plots, you can use the Python library matplotlib.

Task 2 - Classification using Neural Network

In this task, you need to implement a popular machine learning algorithm, Neural Networks. You will train and test a simple neural network with the datasets provided to you and experiment with different settings of hyper parameters.

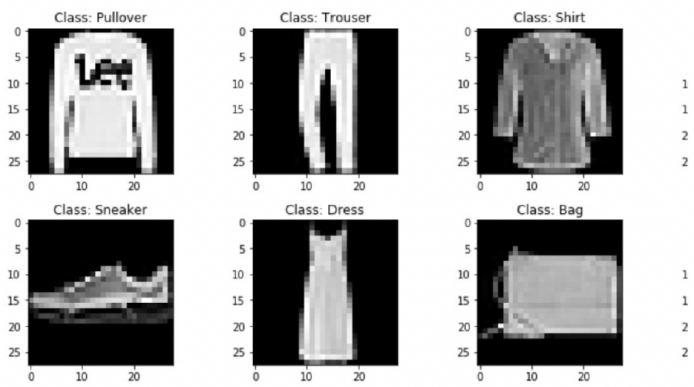
The dataset provided to you is MNIST-like dataset of 28x28 labelled fashion images. A sample of the dataset is given below.



The training and test datasets are provided to you together with this assignment in the following files:
Training examples: `fashion-mnist_train.csv.gz`
Test examples: `fashion-mnist_test.csv.gz`

Data
There are 60,000 training examples and 10,000 test examples. Each example is a grayscale image, associated with a label from 10 classes.
Each image is 28x28 pixels, i.e consists of 784 pixels in total. Each pixel has a grayscale value represented as an integer between 0 and 255, with 0 indicates white and 255 indicates black. The training and test data sets have 785 columns. The first column consists of the class labels (see below) and represents the article of clothing. The rest of the 784 columns contain the pixel-values of the associated image.
Labels
Each training and testing example is assigned to one of the following labels:
0 T-shirt/top
1 Trouser
2 Pullover
3 Dress
4 Coat
5 Sandal
6 Shirt
7 Sneaker
8 Bag
9 Ankle boot

An example of some samples with these classes can be seen below:



Implementation requirements

Your task is to implement a neural network learning algorithm that creates a neural network classifier based on the given training dataset. Your network will have three layers: an input layer, one hidden layer and an output layer.

You should name your main file as **`nn.py`** which accepts **five arguments**. Your code should be run on the command line in the following manner:

```
> python nn.py NInput NHidden NOutput fashion-mnist_train.csv.gz
fashion-mnist_ test.csv.gz
```

Where the meaning of each argument is:
`NInput`: number of neurons in the input layer
`NHidden`: number of neurons in the hidden layer
`NOutput`: number of neurons in the output layer
fashion-mnist_train.csv.gz: the training set
fashion-mnist_test.csv.gz: the test set

Note: you can use numpy.loadtxt() and numpy.savetxt() methods to read from or write to a csv.gz. There is no need to un-compress the file before use. To making plots, you can use the Python library matplotlib.

The activation function used in your neural net should be the basic sigmoid function:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

You will use a version of Back Propagation (BP) algorithm in the training.

For this fashion image recognition assignment, we will encode the output (a number between 0 and 9) by using **10 output neurons**. The neuron with the highest activation will be taken as the prediction of the network. So, the output number y must be represented as a vector of 10 binary digits, all of them being 0 except for the entry at the correct digit.

It is important that your code enable simple and easy adjustment of key parameters such as the number of neurons in the input, hidden and output layers, as well as the number of epochs, mini-batch size and learning rate.

Your code should also run on a small neural network for testing purposes. See below.

You should do the following:

1. Create a neural network of size [784, 30, 10], i.e. 784 neurons in the input layer, 30 neurons in the hidden layer, and 10 neurons in the output layer. Then train it on the training dataset with the following settings: epoch = 30, mini-batch size = 20, η = 3.

   Test your code on the test dataset and draw a graph of **test accuracy vs epoch**. Print the **maximum accuracy** achieved.

2. Train new neural networks with the same settings as in (1.) but with different learning rates η = 0.001, 0.01, 1.0, 10, 100. Plot a graph of **test accuracy vs epoch for each η on the same graph**. Print the **maximum accuracy achieved for each η**. Remember to create a new neural net each time so it starts learning from scratch.

3. Train new neural nets with the same settings as in (1.) above but with different mini-batch sizes = 1, 5, 20, 100, 300. Plot **maximum test accuracy vs mini-batch size**. Which one achieves the maximum test accuracy? Which one is the slowest?

4. Try different hyper-parameter settings (number of epochs, and mini-batch size, etc.). Report the maximum test accuracy you achieved and the settings of all the hyper parameters.

Report Requirements

1. **You should write one report for both tasks, and the report will consist of four parts**: Part 1 (report for Task 1), **Part 2** (report for Task 2), **Appendix 1** (your source code for Task 1), **Appendix 2** (your source code for Task 2).

   The file for the document must be in one of the two formats: Microsoft Word (.doc/.docx), or Adobe PDF (.pdf).

   There is no page limit for the report. But basic criteria of scientific writing should be followed.

   - *Part 1 of your report should include:*
     a. Software design: a detailed description of your code and how it works. Include details on key functions and data structures you are using in your program.
     b. The test results and diagrams, see paragraph (d.) in Task 1.
     c. Discuss the results, include any observations or insights you gained from running tests.
     d. Write a conclusion paragraph that summaries and explains your findings.
   - *Part 2 of your report should include:*
     a. Software design: information about key functions and data structures.

b. All experimental results and graphs mentioned above, and detailed explanations of these results. i.e. try to explain the logic behind those results.
c. Write a conclusion paragraph that summaries and explains your findings.

Submission

You should submit the following documents through the link provided in L@G by the due date:

1. A copy of your report as specified above. Name the file: *firstname_surname_Snumbe*r_A2_report (for example, Bart_Simpson_s123456_A2_report.pdf)
2. A zip file that includes your code (.py files). Name the file: *firstname_surname_Snumbe*r_A2_code.zip (for example, Bart_Simpson_s123456_A2_code.zip)

Note: Make sure your code is well documented and include comments to explain key parts.

The focus of this assignment is about understanding basic concepts and techniques in machine learning. As the dataset for training is relatively small, the accuracy for each algorithm may not be high, but you are encouraged to achieve a relatively high accuracy as you can.

Marking Scheme

A detailed marking rubric is provided with this document in L@G.

****** End of Assignment Description ******